

Lezione 22 parte 4 - AOP(Aspect oriented Programming)

AOP – Introduzione

Durante il modulo di Java abbiamo esplorato a fondo l'OOP (Object-Oriented Programming): classi, oggetti, ereditarietà, incapsulamento.

Con Spring abbiamo poi visto come questi concetti si traducono in un'architettura a strati — Controller, Service, DAO — tenuta insieme dalla Dependency Injection.

L'AOP (Aspect-Oriented Programming):

- **non è un paradigma alternativo all'OOP, ma una sua estensione.**
Nasce dall'osservazione che, nonostante i grandi vantaggi dell'OOP, nel tempo sono emerse alcune carenze strutturali.

Il problema centrale è il seguente: certi comportamenti — come il logging, la gestione delle eccezioni, il controllo degli accessi o il tracciamento delle statistiche — tendono a **ripetersi trasversalmente** in molte classi e metodi.

In OOP, classe e oggetto non sono strumenti adatti a modellare questo tipo di logica "orizzontale":

- **il rischio è duplicare codice ovunque, violando il principio di responsabilità singola.**

L'AOP risolve questo problema introducendo il concetto di **aspetto (aspect)**:

- **un modulo separato che racchiude la logica trasversale e si "aggancia" automaticamente ai punti del codice che ci interessano, senza che le classi coinvolte ne siano consapevoli.**

✓ I benefici pratici sono immediati:

- **Progettazione** più pulita, perché ogni classe mantiene una sola responsabilità
- **Sviluppo** più rapido, perché la logica trasversale si scrive una volta sola
- **Manutenzione** semplificata, perché modificare un comportamento trasversale significa toccare un solo punto

- **Evoluzione** più sicura, perché aggiungere nuovi aspetti non richiede di modificare il codice esistente

Cos'è un Aspetto

Un **aspetto** è:

- un modulo che incapsula un comportamento trasversale, detto in inglese *cross-cutting concern*:
 - una funzionalità che non appartiene logicamente a una singola classe, attraversa più componenti del sistema e si applica in punti diversi dell'esecuzione del programma.

Un aspetto descrive **cosa fare e in quali condizioni farlo**, ma non stabilisce il *dove* nel codice sorgente — è il framework AOP a occuparsi di questo.

Il problema: il logging come esempio concreto

Pensa a tutto il codice scritto finora: ogni volta che volevamo monitorare l'esecuzione di un metodo, aggiungevamo una stampa o usavamo la classe `Logger`.

Quella logica di tracciamento era **trasversale** — **si ripeteva in classi diverse, senza appartenere davvero a nessuna di esse.**

Il risultato era codice "sporcato" da responsabilità che non gli competevano.

L'OOP tradizionale suggerirebbe di creare una classe dedicata al logging. Ma a quel punto sorge il problema: **come fa quella classe a sapere quando e dove intervenire?**

Le altre classi dovrebbero conoscerla e chiamarla esplicitamente, creando accoppiamento e di nuovo duplicazione.

L'AOP risolve questo nodo alla radice:

- **l'aspetto viene definito una volta sola, e il framework lo intreccia automaticamente con il codice nei punti stabiliti, senza che le classi coinvolte ne sappiano nulla.**

Perché l'OOP non basta

L'OOP si fonda su un principio chiave:

- ogni comportamento appartiene a un oggetto.
È proprio qui che emerge il limite:
- I *cross-cutting concern* — come il logging, la gestione della sicurezza o il tracciamento delle statistiche — non hanno un oggetto naturale di riferimento.
 - Non appartengono a una classe in particolare, ma riguardano potenzialmente tutte le classi del sistema.

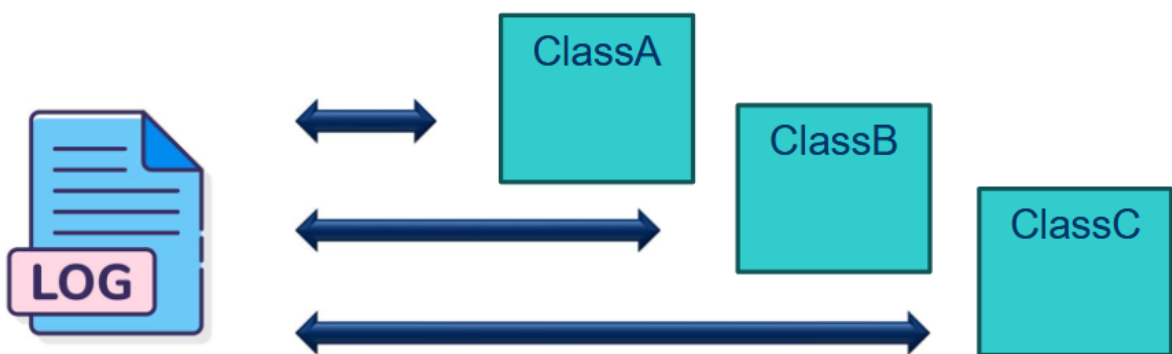
≡ Riprendendo l'esempio del logging:

non è responsabilità del Controller loggare, né del Service, né del DAO. Eppure tutti ne hanno bisogno.

In OOP puro, l'unica soluzione sarebbe creare una classe `Logger` e richiamarla esplicitamente ovunque — ma questo significa accoppiamento, ripetizione e codice sporcato di logica che non compete a nessuno strato.

L'AOP nasce esattamente per rispondere a questa carenza:

- offre un modo per modellare i concern trasversali fuori dalle classi di business, mantenendo ogni componente focalizzato sulla propria responsabilità.



Separation of Concerns

Il principio di **Separation of Concerns** afferma che:

- è auspicabile separare le diverse questioni — i diversi *concern* — che caratterizzano un sistema, in modo da poterle analizzare e gestire singolarmente.

Detto questo, nella realtà gli aspetti sono spesso interconnessi e intrecciati:

- non possono essere valutati in completo isolamento, perché agiscono sulle stesse componenti del sistema.
- È proprio da questa tensione — separare ma anche incrociare — che nasce l'esigenza della programmazione orientata agli aspetti .

In pratica, il modello AOP introduce due categorie di classi:

1. Le classi target:

- sono le classi di business, come i ``Service``.
- Rappresentano la logica principale del sistema e si sviluppano orizzontalmente nell'architettura.

2. Le classi aspetto:

- modellano i concern trasversali — logging, statistiche, sicurezza — e si sviluppano verticalmente, incrociando più classi target contemporaneamente.

Questa separazione permette di:

- mantenere ogni classe target focalizzata sulla propria responsabilità,
- mentre gli aspetti vengono applicati trasversalmente dal framework, senza che i target ne siano consapevoli.

☞ Si programmano aspetti **SOLO** per gestire questo tipo di situazioni trasversali — **NON per logica di business ordinaria.**



Cross-Cutting Concerns

I **cross-cutting concern** sono:

- **quelle caratteristiche funzionali di un applicativo che attraversano trasversalmente la logica di business.**

Non appartengono a un singolo strato o a una singola classe, ma riguardano l'intero sistema.

☰ **Servizi Infrastrutturali: esempi classici**

1. **il logging,**
2. **la sicurezza,**
3. **la gestione della concorrenza,**
4. **il tracciamento delle statistiche.**

Nessuno di questi è logica di business — non calcolano nulla, non gestiscono dati del dominio — eppure sono necessari ovunque.

Affrontare questi concern con l'OOP classico porta inevitabilmente a due problemi:

× **Failure**

- **Duplicazione del codice:**
 - **la stessa logica — ad esempio una chiamata al Logger — viene riscritta in ogni classe che ne ha bisogno.**
- **Frammentazione del codice:**
 - **la logica trasversale si sparpaglia in punti diversi del sistema, rendendo difficile trovarla, modificarla o rimuoverla.**

In entrambi i casi il risultato è lo stesso:

- **le classi di business vengono inquinate da responsabilità che non gli competono, violando il Single Responsibility Principle che abbiamo già incontrato studiando i principi SOLID.**

L'AOP risolve entrambi i problemi raccogliendo ogni concern trasversale in un unico aspetto, applicato automaticamente nei punti stabiliti.

🔗 **I Principi SOLID**

I principi **SOLID** sono 5 linee guida fondamentali della progettazione object-oriented, formulate da Robert C. Martin.

Il loro obiettivo è rendere il software più manutenibile, estensibile e leggibile nel tempo.

Le cinque lettere corrispondono a:

1. **S** – *Single Responsibility Principle*: **una classe deve avere una sola ragione per cambiare, ovvero una sola responsabilità.**

≡ Immaginiamo di avere una classe

`OperazioneService` :

calcola le operazioni algebriche.

Se aggiungessimo anche il logging e il conteggio delle statistiche dentro di lei, avrebbe tre responsabilità distinte.

Ogni responsabilità va in una classe separata.

× Esempio errato

Aggiungiamo quindi, oltre al calcolo della somma algebrica, anche il conteggio delle statistiche.



Java

```
1 // Sbagliato: troppe responsabilità
2 @Service
3 public class OperazioneService {
4     public int somma(int a, int b) {
5         System.out.println("Chiamata a somma"); //
        logging
6         contatoreSomma++; // statistica return
7         a + b; // calcolo
8     }
9 }
```

Questo viola la *Single Responsibility Principle*: aggiungiamo troppe responsabilità ad un'unica classe.

✓ Esempio corretto:

Suddividiamo le classi in base alla loro responsabilità

1. Classe Service OperazioneService



Java

```
1 // ✅ Corretto: una sola responsabilità per classe
2 @Service
3 public class OperazioneService {
4     public int somma(int a, int b) {
5         return a + b;
6     }
7 }
```

Essendo questa una classe Service, questo ha perfettamente senso, perché:

- Deve contenere solo la logica di business ad alto livello
- Questa operazione non è né coordinamento (Controller) né, tantomeno, accesso ai dati (DAO).

Quindi è giusto che la somma algebrica stia dentro una classe di servizio.

2. Classe Bean StatisticheComponent



Java

```
1 @Component
2 public class StatisticheComponent {
3     private int contatoreSomma = 0;
4     public void incrementaSomma() {
5         contatoreSomma++;
6     }
7     public int getContatoreSomma() {
8         return contatoreSomma;
9     }
10 }
11
```

Notare come questa classe è annotata con '@Component'. Questa classe contiene, difatti, il contatore della somma e i metodi per incrementare il contatore e ritornarlo.

2. O – Open/Closed Principle: una classe deve essere aperta all'estensione ma chiusa alla modifica.

≡ **Esempio: Classe** Sconto >

Supponiamo di avere una classe `Sconto` che implementa un blocco `if/else` per ogni tipo di sconto, quindi si modifica la classe ogni volta che si aggiunge un nuovo tipo di sconto.

In questo caso è meglio definire un'interfaccia e creare sotto classi concrete che rappresentano una nuova implementazione per ogni tipo di sconto — senza toccare il codice esistente.

× **Esempio errato**



Java

```
1  >public class Sconto {
2  >   public double calcola(String tipo, double prezzo)
3  >   {
4  >       if (tipo.equals("studente")) {
5  >           return prezzo * 0.80;
6  >       } else if (tipo.equals("anziano")) {
7  >           return prezzo * 0.75;
8  >       }
9  >       return prezzo;
10 >   }
11 >}
12 >'''
13 > Questo esempio viola il principio open/closed,
14 > poiché ogni volta che si vuole aggiungere un nuovo
15 > tipo di sconto si deve modificare il codice del metodo
16 > 'calcola()'.
17 >
18 >[!done] **Esempio Corretto:**
19 >'''java
20 >[✓] Corretto: aperta all'estensione, chiusa alla
21 > modifica public interface Sconto {
22 >   double calcola(double prezzo);
23 >}
24 >public class ScontoStudente implements Sconto {
25 >   public double calcola(double prezzo) {
26 >       return prezzo * 0.80;
27 >   }
28 >}
```



```

23 >}
24 >public class ScontoAnziano implements Sconto {
25 >    public double calcola(double prezzo) {
26 >        return prezzo * 0.75;
27 >    }
28 >}
29 >```
30 > Questo esempio segue alla perfezione questo
    principio: le sottoclassi implementano l'interfaccia
    sconto la quale dichiara un metodo astratto comune per
    ogni sotto classe, senza che si debba andare a
    cambiare il codice dell'interfaccia poiché essa è
    perfettamente estendibile alle sue sotto classi.
31 > Quindi per ogni sotto tipologia di sconto verrà
    specializzato il metodo `calcola()` senza però che
    questo venga modificato e cambiato ogni volta.

```

3. L – Liskov Substitution Principle: **una sottoclasse deve poter sostituire la sua superclasse senza alterare il comportamento del programma.**

☰ **Esempio: La classe** Quadrato **che estende la super** >
classe Rettangolo

Ipotizziamo di avere due classi: una sotto classe Quadrato che estende Rettangolo, ma sovrascrive i setter per mantenere i lati uguali.

Chi usa un Rettangolo non si aspetta questo comportamento — la sottoclasse rompe il contratto della superclasse.

× Esempio Errato



Java

```

1 // ✗ Sbagliato: Quadrato rompe il contratto di
  Rettangolo public class Rettangolo {
2     protected int larghezza;
3     protected int altezza;
4     public void setLarghezza(int larghezza) {
5         this.larghezza = larghezza;
6     }

```

```

7      public void setAltezza(int altezza) {
8          this.altezza = altezza;
9      }
10     public int calcolaArea() {
11         return larghezza * altezza;
12     }
13 }
14
15 public class Quadrato extends Rettangolo {
16     @Override public void setLarghezza(int
larghezza) {
17         this.larghezza = larghezza;
18         this.altezza = larghezza; // effetto
collaterale inatteso!
19     }
20     @Override public void setAltezza(int altezza)
{
21         this.altezza = altezza;
22         this.larghezza = altezza; // effetto
collaterale inatteso!
23     }
24 }

```

Quindi la sotto classe override i setter della super classe, così facendo però sovrascrive il comportamento della classe padre rompendo il programma

✓ Esempio corretto:



Java

```

1  // ✓ Corretto: modella le forme come concetti
   separati public
2  interface Forma {
3      int calcolaArea();
4  }
5  public class Rettangolo implements Forma {
6      private int larghezza;
7      private int altezza;
8      public Rettangolo(int larghezza, int altezza)
{
9          this.larghezza = larghezza;

```

```

10         this.altezza = altezza;
11     }
12     public int calcolaArea() {
13         return larghezza * altezza;
14     }
15 }
16
17 public class Quadrato implements Forma {
18     private int lato;
19     public Quadrato(int lato) {
20         this.lato = lato;
21     }
22
23     public int calcolaArea() {
24         return lato * lato;
25     }
26 }

```

Anche in questo caso abbiamo definito una interfaccia con un metodo comune a tutte le sue sotto classi: questo metodo poi lo andiamo a specializzare per ciascuna sotto classe, in questo modo ogni sotto classe calcola l'area in modo diverso e di conseguenza il metodo della super classe(in questo caso dell'interfaccia) non viene overrideato e quindi il comportamento del programma non viene alterato.

4. I – Interface Segregation Principle: **è preferibile avere molte interfacce specifiche piuttosto che una sola interfaccia generica.**

☰ **Esempio: interfaccia** Animale >

Supponiamo di avere una interfaccia `Animale` con i metodi `mangia()`, `vola()` e `nuota()`.

Ora abbiamo anche una classe concreta `Cane` che implementa l'interfaccia ma così facendo costringiamo questa classe ad implementare metodi che non gli competono.

Per risolvere questo problema una buona pratica spezzare l'interfaccia `Animale` in interfacce più piccole.

✗ **Esempio errato**



Java

```
1 // ✗ Sbagliato: interfaccia troppo generica
2 public interface Animale {
3     void mangia();
4     void vola();
5     void nuota();
6 }
7 public class Cane implements Animale {
8     public void mangia() { System.out.println("Il
cane mangia");}
9     public void vola() { /* non implementato, non
ha senso */ }
10    public void nuota() { System.out.println("Il
cane nuota"); }
11 }
```

Come possiamo notare da questo esempio la classe `Cane` è costretta a implementare metodi totalmente errati a livello logico per un Cane (es: `vola()`)

✓ Esempio corretto



Java

```
1 // ✓ Corretto: interfacce specifiche per capacità
2 public interface Mangiatore {
3     void mangia();
4 }
5 public interface Volatore {
6     void vola();
7 }
8 public interface Nuotatore {
9     void nuota();
10 }
11 public class Cane implements Mangiatore, Nuotatore
{
12     public void mangia() { System.out.println("Il
cane mangia"); }
13     public void nuota() { System.out.println("Il
cane nuota"); }
```

```

14     }
15     public class Aquila implements Mangiatore,
        Volatore {
16         public void mangia() {
            System.out.println("L'aquila mangia"); }
17         public void vola() {
            System.out.println("L'aquila vola"); }
18     }

```

Spezzando l'interfaccia `Animale` in altre interfacce più piccole possiamo implementare i comportamenti corretti per ciascuna classe concreta.

Questo è il principio *Interface Segregation* in azione.

5. **D – Dependency Inversion Principle:** i moduli di alto livello non devono dipendere da quelli di basso livello; entrambi devono dipendere da astrazioni.

≡ **Esempio:** `OperazioneController` >

Questo principio, senza saperlo, l'abbiamo già riscontrato con Spring.

Supponiamo di avere una classe una classe Controller, `OperazioneController`, che come attributo un oggetto dell'interfaccia `OperazioneService`.

Come abbiamo già visto `OperazioneController` non istanzia direttamente `OperazioneService` con `new` ma la riceve tramite l'annotation `'@Autowired'`: - questa cosa dipende dall'astrazione gestita dal container Spring, non dall'implementazione concreta.

× Esempio Errato



Java

```
1 // ✗ Sbagliato: dipendenza diretta
  dall'implementazione concreta
2 public class OperazioneController {
3     private OperazioneService service = new
  OperazioneService(); // accoppiamento forte
4 }
```

✓ Esempio corretto



Java

```
1 // ✓ Corretto: dipendenza dall'astrazione,
  gestita da Spring
2 @RestController
3 @RequestMapping("/operazioni")
4 public class OperazioneController {
5     @Autowired
6     private OperazioneService service; // Spring
  inietta l'implementazione
7 }
```

In questo caso, tramite l'annotazione `@Autowired`, Spring inietta (cioè istanzia) la classe concreta della Service a partire dalla reference dell'interfaccia della Service.

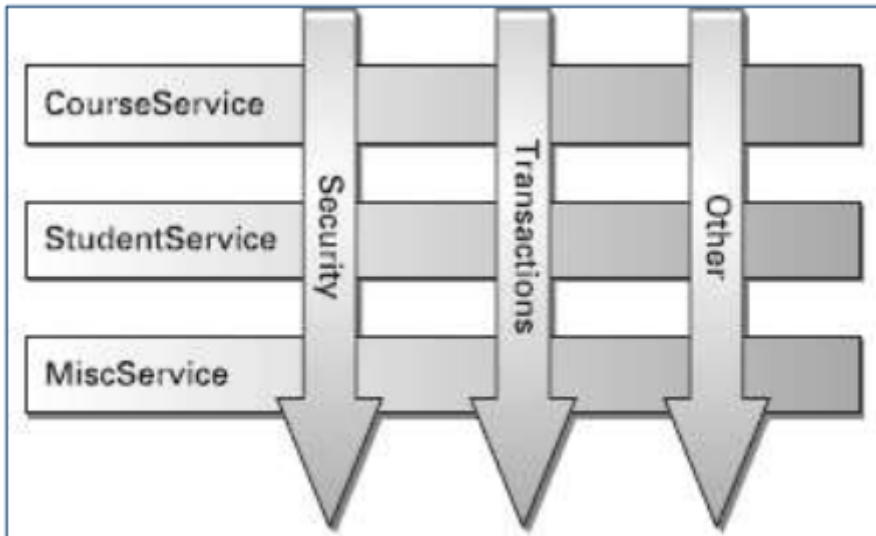
Questo è *Dependency Inversion Principle* in azione.

Nel contesto dell'AOP, il principio più rilevante è l'**SRP**:

- inserire logica trasversale come il logging direttamente nelle classi di business significa assegnare a quella classe più di una responsabilità, rendendo il codice più difficile da mantenere e modificare.

Dimostrazione Cross-Cutting Concerns

Per comprendere meglio questo concetto guardiamo l'immagine:



Rappresenta visivamente il concetto di cross-cutting concern in un sistema con tre classi di servizio:

1. CourseService ,
2. StudentService
3. MiscService .

Le frecce **orizzontali** rappresentano i **target** — le classi di business che si sviluppano ciascuna per conto proprio, ognuna con la propria logica.

Le frecce **verticali** — Security, Transactions, Other — rappresentano gli **aspetti**:

- attraversano trasversalmente tutte e tre le classi contemporaneamente, indipendentemente da cosa fa ciascuna di esse.

Il messaggio visivo è immediato:

- **Security non appartiene a CourseService , né a StudentService , né a MiscService — appartiene a tutte e tre allo stesso tempo.**
- **Lo stesso vale per Transactions e per qualsiasi altro concern trasversale.**

In OOP puro, ogni classe dovrebbe gestire questi comportamenti da sola — con il risultato di duplicare la stessa logica in tre posti diversi.

Con l'AOP invece ogni freccia verticale diventa **un unico Aspect, scritto una volta sola, che il framework applica automaticamente su tutti i target coinvolti.**

🕒 **È importante notare che nella realtà un aspetto non attraversa necessariamente tutti i target del sistema.**

Tramite il **Pointcut** — che vedremo a breve — è possibile definire con precisione su quali classi e su quali metodi un aspetto deve essere applicato. Il diagramma è una semplificazione visiva del concetto.

AOP: Terminologia

Aspect

Un **Aspect**:

- rappresenta la modularizzazione di una caratteristica trasversale del sistema — ad esempio il logging, la sicurezza o il tracciamento delle statistiche.
- Ha principalmente un **significato progettuale**:
 - descrive cosa vogliamo separare e isolare, ma non ha una traduzione diretta a livello di codice.
 - È il concetto astratto da cui tutto parte.

Advice

Un **Advice** è:

- l'implementazione effettiva di un Aspect.
- È il codice concreto che viene eseguito quando l'aspetto entra in azione.
- I framework AOP — Spring incluso — implementano gli advice tramite degli interceptor:
 - componenti che si interpongono nell'esecuzione del target e ne filtrano il lavoro.

È importante capire che questo filtraggio lo decidiamo noi, e il *quando* dipende dalla natura del concern:

≡ Example

- il logging:
 - tipicamente si applica *dopo* l'esecuzione del metodo, per tracciarne l'esito
- la sicurezza e l'autenticazione:

- si applicano *prima*, per bloccare l'esecuzione se le condizioni non sono soddisfatte

L'interceptor inoltre non si limita a osservare:

- può anche trattenere o modificare il flusso di esecuzione.

Target

Un Target è:

- il metodo — o la classe — a cui vogliamo applicare uno o più advice.
- È più preciso dire che il target è un metodo specifico di una classe, non la classe nel suo insieme, anche se per convenzione si parla spesso della classe come target.

Pointcut

Un Pointcut è un'espressione che:

- individua tutti i metodi su cui deve agire un advice.
- È il meccanismo con cui diciamo al framework *dove* applicare l'aspetto:
 - su quali classi, su quali metodi, con quali condizioni.
- Come abbiamo visto, un pointcut può essere selettivo — non deve necessariamente coprire tutti i target del sistema.

JoinPoint

Un JoinPoint è:

- il punto del workflow in cui l'advice si incrocia con il target.

In altre parole:

- è il momento preciso in cui l'aspetto entra in azione durante l'esecuzione del programma.

Può corrispondere a:

- la chiamata a un metodo
- il sollevamento di un'eccezione
- il cambiamento di stato di un attributo

Il modulo AOP di Spring prevede advice che agiscono esclusivamente sui metodi, e li classifica in base al *quando* rispetto all'esecuzione del target:

- **Before** – prima dell'esecuzione del metodo
- **After** – dopo l'esecuzione del metodo
- **Around** – sia prima che dopo, con controllo completo sul flusso

AOP: I Tipi di JoinPoint in Spring

Spring mette a disposizione 5 tipi di joinpoint, ciascuno dei quali dà vita a un corrispondente tipo di interceptor:

1. **Before** — l'interceptor scatta prima che il metodo del target vada in esecuzione.

≡ Tipico caso d'uso:

la sicurezza e l'autenticazione, dove vogliamo verificare le condizioni prima di permettere l'esecuzione.

2. **After** — l'interceptor scatta dopo che il metodo del target va in esecuzione, indipendentemente dall'esito — che sia andato a buon fine o che abbia sollevato un'eccezione.

≡ Utile per operazioni di cleanup o rilascio di risorse.

3. **After Returning** — l'interceptor scatta dopo l'esecuzione del metodo, ma solo se ha avuto successo.

≡ Tipico caso d'uso:

il logging del risultato di un'operazione andata a buon fine.

4. **After Throwing** — l'interceptor scatta dopo l'esecuzione del metodo, ma solo se è stata sollevata un'eccezione.

≡ Tipico caso d'uso:

il logging degli errori o la notifica di un fallimento.

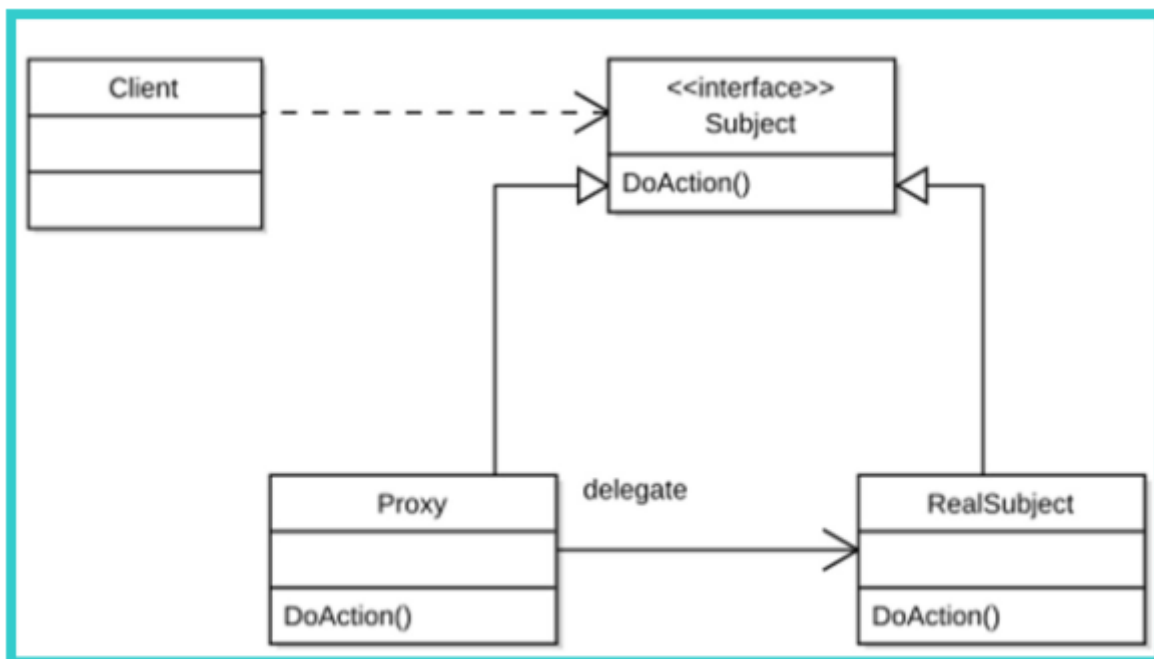
5. **Around** — l'interceptor scatta sia prima che dopo l'esecuzione del metodo. Combina le caratteristiche di Before e After, ed è il tipo più potente perché ha controllo completo sul flusso — può decidere se far proseguire l'esecuzione del target oppure no.

Weaving e Proxy

Il Design Pattern Proxy

Per capire come Spring applica concretamente gli aspetti, bisogna prima richiamare il **Design Pattern Proxy**.

Il diagramma UML mostra la struttura:



Pattern proxy

il Client non parla direttamente con il RealSubject — il componente che contiene la logica reale — ma con un Proxy che implementa la stessa interfaccia Subject. Il Proxy si **spaccia per l'oggetto reale** agli occhi del client, ma internamente delega la chiamata al RealSubject, potendo intervenire prima, dopo o intorno all'esecuzione.

Nel contesto AOP:

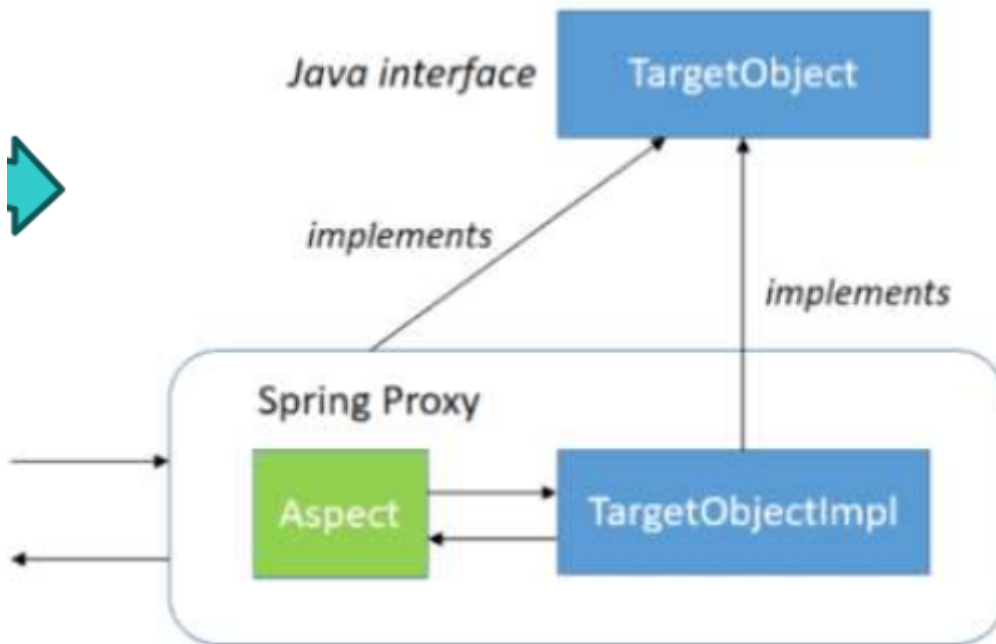
- il RealSubject corrisponde al **Target** — la classe di business
- il Proxy corrisponde all'**Advice** — il componente che intercetta e aggiunge il comportamento trasversale

Weaving

Il **Weaving** è:

- il processo concreto con cui un framework AOP applica gli **advice** alle **classi target** nei **joinpoint** stabiliti, secondo la definizione dei **pointcut**.

Spring realizza il weaving **a runtime** — come mostra questo diagramma:



creando un **Spring Proxy** che avvolge il **TargetObjectImpl**.

Quando il client chiama un metodo del target, in realtà sta chiamando il proxy: **è lui che coordina l'esecuzione dell'Aspect e del TargetObjectImpl, senza che il target sappia nulla di ciò che lo circonda.**

Entrambi — Proxy e **TargetObjectImpl** :

- **implementano la stessa interfaccia Java (TargetObject), garantendo che il client non si accorga della differenza.**

In termini pratici il weaving funziona così:

- quando l'applicazione si avvia, Spring legge le annotazioni **@Aspect**, **@Before**, **@After** e i **pointcut** definiti.
- A quel punto costruisce automaticamente il proxy — che rappresenta il punto di intreccio tra la logica di business del target e il comportamento trasversale dell'aspect.
- Da quel momento in poi ogni chiamata al target passa attraverso il proxy, che coordina target e advice nell'ordine stabilito.

📅 In sintesi il weaving è l'applicazione pratica del Design Pattern Proxy nel contesto AOP:

- il Target viene avvolto dal proxy, l'Aspect viene iniettato al suo interno, e il client interagisce sempre e solo con il proxy — in modo completamente trasparente.

≡ Analogia: il casello autostradale

Per capire concretamente il weaving, immagina un **casello autostradale**:

- la macchina è il **Client** — percorre la strada senza sapere cosa lo aspetta
- l'autostrada è il **Target** — contiene la logica di business e prosegue dritta, ignara del casello.
- il casello nella sua interezza è il **Proxy** — si inserisce in un punto preciso del percorso e intercetta il transito
- il concetto del pedaggio è l'**Aspect** — l'idea astratta del concern trasversale
- lo sportello che riscuote è l'**Advice** — l'implementazione concreta dell'Aspect
- il punto geografico preciso del casello è il **JoinPoint** — il momento esatto in cui il proxy intercetta la chiamata
- la regola "tutti i veicoli che percorrono questa tratta devono fermarsi" è il **Pointcut** — l'espressione che stabilisce dove e quando l'advice si applica

La cosa fondamentale è che l'autostrada non sa dell'esistenza del casello — è il casello che si inserisce nel percorso, non il contrario.

Esattamente come il Target non sa nulla dell'Aspect che lo circonda.

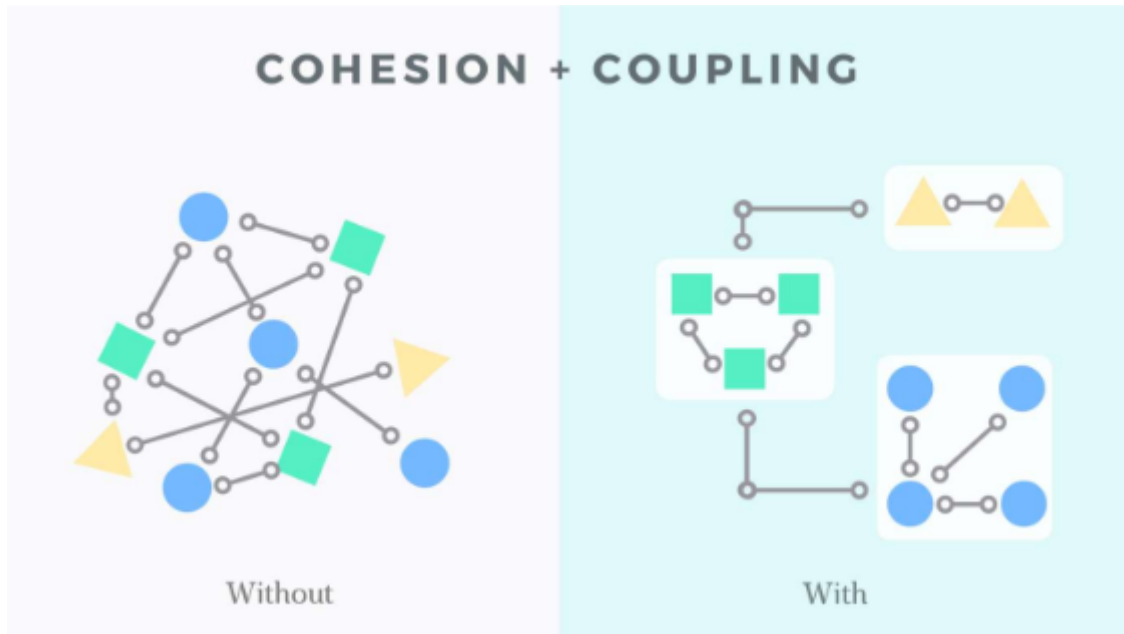
🔔 **Nota importante: il flusso è completamente sincrono — la macchina si ferma, paga, e solo dopo prosegue.**

L'AOP è trasversale rispetto alla struttura del codice, ma non introduce alcuna forma di esecuzione asincrona o parallela.

AOP: Vantaggi

High Cohesion, Low Coupling

Il diagramma mostra visivamente il risultato concreto dell'applicazione dell'AOP — e più in generale di una buona architettura — attraverso due concetti fondamentali: **Cohesion** e **Coupling**.



Nel lato **Without**:

- i componenti del sistema sono tutti interconnessi tra loro in modo caotico:
 - ogni elemento conosce e dipende da molti altri.
 - Questo è il risultato di un sistema senza separazione delle responsabilità — logica di business, logging, sicurezza e statistiche mescolati insieme ovunque.
 - È difficile da leggere, modificare e testare.

Nel lato **With**:

- i componenti sono raggruppati in moduli coesi con poche connessioni tra loro.
- Ogni gruppo si occupa di una cosa sola, e i gruppi comunicano tra loro solo dove necessario.

Questo corrisponde esattamente a due obiettivi architetturali che l'AOP aiuta a raggiungere:

- High Cohesion — ogni classe e ogni modulo ha una responsabilità ben definita e concentrata.

☰ **La** OperazioneService **fa solo calcoli, l'Aspect fa solo logging,**
il StatisticheComponent **tiene solo i contatori.**

- **Low Coupling** — **le classi di business non sanno nulla degli aspetti che le circondano.**
 - Non li importano, non li chiamano, non dipendono da loro.
 - Il framework applica tutto in modo trasparente tramite il weaving.

📋 I vantaggi in sintesi

L'AOP permette di modellare i concern trasversali in classi apposite e di applicarne i comportamenti alle classi target in modo **dichiarativo** — cioè tramite annotazioni e pointcut, senza toccare il codice del target.

I benefici diretti sono due:

- **Elimina la duplicazione** del codice trasversale:
 - **la logica di logging, sicurezza o tracciamento si scrive una volta sola nell'Aspect, non in ogni classe che ne ha bisogno.**
- **Rende la logica trasversale indipendente** dalla logica di business:
 - **si può aggiungere, modificare o rimuovere un aspetto senza toccare nessuna delle classi target.**

AOP: Esempio — Il Logging del Cavaliere

Il problema: dipendenza forte

Ipotizziamo di avere un progetto che modella un **Cavaliere della Tavola Rotonda** che parte per la missione, e un **Menestrello** che ne deve narrare le gesta — una metafora perfetta per il logging.

Un primo approccio OOP classico potrebbe essere questo: il Cavaliere porta il Menestrello come **attributo**, e lo chiama direttamente dentro il metodo `partiPerMissione()`.

java



Java

```
1 public class CavaliereTavolaRotonda {
```

```

2     private String nome;
3     private Menestrello menestrello;
4
5     public void setMenestrello(Menestrello m) {
6         this.menestrello = m;
7     }
8
9     public SantoGaal partiPerMissione() {
10        menestrello.cantaGesta(this.nome + " parte per la
missione");
11        return new SantoGaal();
12    }
13 }

```



Java

```

1 public class Menestrello {
2     public void cantaGesta(String verso) {
3         System.out.println(verso);
4     }
5 }

```

Perché questo modello è sbagliato

Questo approccio funziona, ma viola tutto quello che abbiamo detto finora.

Il Cavaliere ha una **dipendenza forte** dal Menestrello:

- **lo conosce, lo porta con sé, lo chiama esplicitamente.**

Il Cavaliere non dovrebbe sapere nulla del Menestrello — narrare le gesta non è responsabilità sua, è un concern trasversale.

In termini architetturali:

- il **Cavaliere** è il **Target** — **contiene la logica di business**
- il **Menestrello** è l'**Aspect** — **contiene la logica trasversale di logging**

🚫 Un Target non dovrebbe MAI conoscere i propri Aspect.

Il problema nel dettaglio

La soluzione OOP vista prima presenta due criticità concrete:

1. il Cavaliere deve **interrompere la propria missione** per chiamare il Menestrello.

- **La logica di business — partire per la missione e trovare il Graal — viene spezzata da una chiamata a un servizio trasversale che non gli compete.**
- **Il processo di business viene interrotto per invocare un servizio di sistema .**



processo di business interrotto per invocare un servizio di sistema

2. problema di **scalabilità**:

- se nel sistema avessimo altri personaggi — un altro cavaliere, un paladino, un arciere — ognuno di loro dovrebbe portarsi dietro il proprio Menestrello.
- **Il risultato sarebbe:**
 - **tanti Menestrelli istanziati, uno per ogni personaggio**
 - **ogni personaggio accoppiato al proprio Menestrello, con una dipendenza forte e ripetuta ovunque**

La soluzione AOP

Con l'AOP il modello cambia radicalmente:

- **il Menestrello è uno solo — un unico Aspect condiviso da tutto il sistema**
- **il Menestrello non è legato a nessun personaggio — non è attribuito di nessuno, non viene chiamato da nessuno esplicitamente**
- **il Menestrello incrocia la sua vita con quella dei personaggi nei punti stabiliti dal pointcut, tramite il weaving di Spring**
- **tutti i personaggi non sanno dell'esistenza del Menestrello — il Target ignora completamente i propri Aspect**

🔗 **Questo è il cuore dell'AOP:**

separare chi fa la logica di business da chi la osserva, la traccia o la intercetta — senza che i due si conoscano.

AOP: La Soluzione in Codice

Definizione dell'Advice

Modelliamo il Menestrello come un **Aspect**:

- non più una classe con dipendenza forte, ma un componente autonomo annotato con `@Component` e `@Aspect`



Java

```
1  @Component
2  @Aspect
3  public class Menestrello {
4
5      @Before("execution(*
        tavola.CavaliereTavolaRotonda.partiPerMissione(..))")
6      public void cantaGesta() {
7          System.out.println("il cavaliere è in partenza..");
8      }
9  }
```

Analizziamo le annotazioni:

- `@Component` — rende il Menestrello un bean gestito da Spring
- `@Aspect` — dichiara che questa classe è un Aspect AOP
- `@Before` — definisce il **tipo di joinpoint**: l'interceptor scatta *prima* dell'esecuzione del metodo target
- La stringa `execution(* tavola.CavaliereTavolaRotonda.partiPerMissione(..))` è il **pointcut**:
 - **individua esattamente il metodo su cui l'advice deve agire.**

La sintassi è:



Java

```
1  execution(* package.ClasseTarget.metodoTarget(..))
```

dove * indica qualsiasi tipo di ritorno e (...) indica qualsiasi lista di parametri.

Il risultato: il Cavaliere libero dal Menestrello

A questo punto la classe `CavaliereTavolaRotonda` può essere riscritta eliminando **qualsiasi riferimento** al Menestrello:

```
Java

1 public class CavaliereTavolaRotonda {
2     private String nome;
3
4     public SantoGaal partiPerMissione() {
5         return new SantoGaal();
6     }
7 }
```

Il Cavaliere non sa nulla dell'esistenza del Menestrello, non lo importa, non lo chiama.

Si occupa esclusivamente della propria logica di business — trovare il Graal.

È Spring che, tramite il **weaving a runtime**, fa sì che il Menestrello intercetti la chiamata a `partiPerMissione()` nel momento giusto, avvolgendo il Target con il proprio Proxy.

Non esiste più alcun legame diretto tra il servizio trasversale e la logica di business.



L'intercettore (Advice) circonda il Target

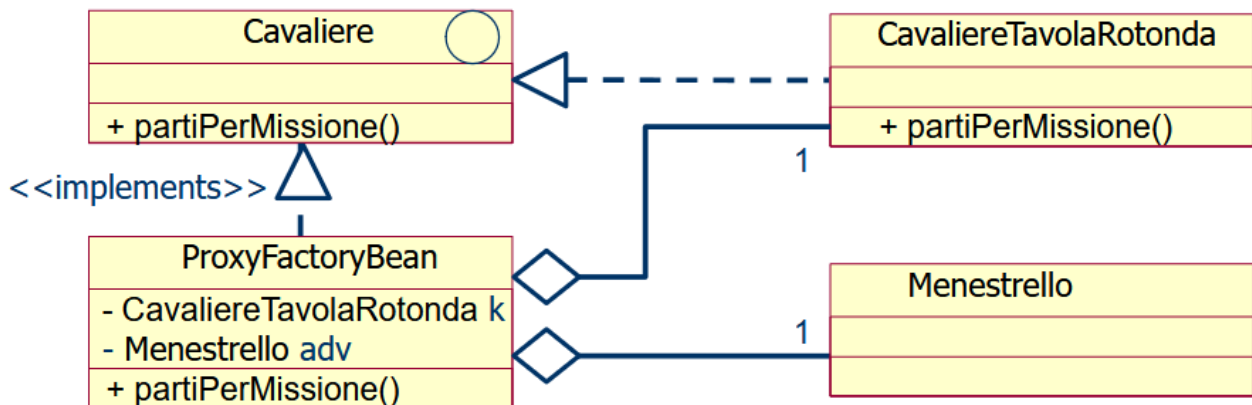
AOP: Il Proxy a Runtime

Come Spring collega Target e Advice

Il motore AOP di Spring collega Target e Advice creando un Proxy a runtime.

Questo avviene tramite la classe `ProxyFactoryBean`, **che agisce esattamente come il Design Pattern Proxy che abbiamo già visto.**

Il diagramma UML mostra la struttura nel caso in cui il target implementi una interfaccia — che è il caso più comune in Spring:

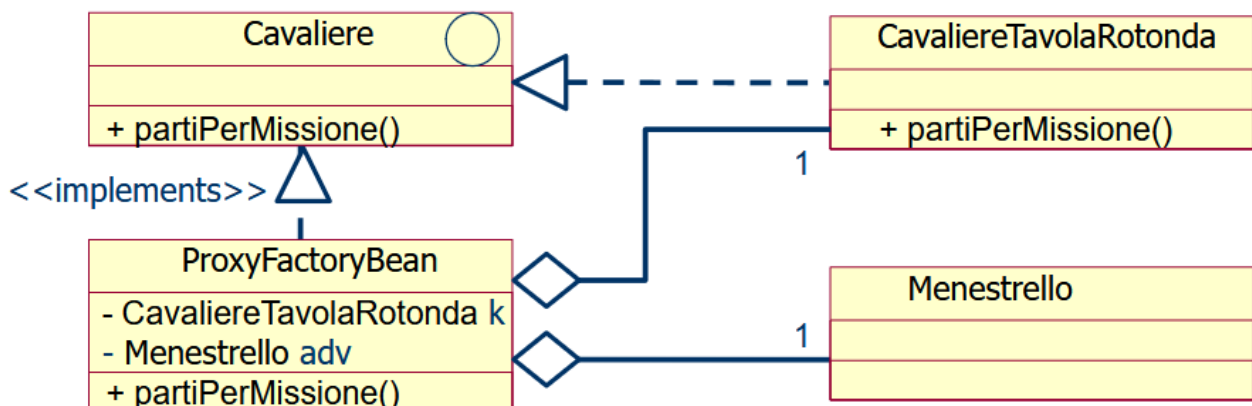


- `Cavalere` è **l'interfaccia Java che dichiara il metodo `partiPerMissione()`**
- `CavalereTavolaRotonda` **è l'implementazione concreta — il Target — che implementa `Cavalere`**
- `ProxyFactoryBean` implementa anch'essa `Cavalere`, e al suo interno possiede:
 - **un'istanza di `CavalereTavolaRotonda` — il Target**
 - **un'istanza di `Menestrello` — l'Advice**

Il flusso di esecuzione

Quando il client chiama `partiPerMissione()`, in realtà sta chiamando il metodo della `ProxyFactoryBean` — non direttamente il `Cavalere`.

È la `ProxyFactoryBean` a decidere, in base al tipo di advice configurato, l'ordine di esecuzione:



Il diagramma chiarisce le relazioni tra i componenti con precisione:

1. Cavaliere **è l'interfaccia che dichiara il metodo comune** `partiPerMissione()`.
2. CavaliereTavolaRotonda **realizza l'interfaccia Cavaliere** — **relazione rappresentata in UML con una freccia tratteggiata con punta triangolare vuota, che in Java corrisponde a** `implements` **— e specializza il metodo con la propria implementazione concreta.**

ProxyFactoryBean implementa anch'essa l'interfaccia Cavaliere, e al suo interno **aggrega** due attributi tramite composizione:

- `-CavaliereTavolaRotonda k` — **dove** `k` **sta per** *knight*, il Target
- `-Menestrello adv` — **dove** `adv` **sta per** *advice*, l'Aspect

🔗 I rombi vuoti nel diagramma, come sappiamo, indicano una **aggregazione o responsabilità**:

- **In questo caso** ProxyFactoryBean **referenzia i due oggetti, ma non ne è responsabile del ciclo di vita** — **se viene distrutta, `k` e `adv` continuano ad esistere.**

Il punto chiave è che sia CavaliereTavolaRotonda che ProxyFactoryBean implementano la stessa interfaccia Cavaliere: questo rende il proxy **trasparente** al client, che vede solo il contratto dell'interfaccia senza sapere che in mezzo c'è un proxy.

Implementazione in Java

1. Interfaccia Cavaliere



Java

```
1 // Interfaccia comune
2 public interface Cavaliere {
3     SantoGaal partiPerMissione();
4 }
```

2. Classe Concreta CavaliereTavolaRotonda (Target)



Java

```
1 // Target - implementa l'interfaccia
```

```

2  public class CavaliereTavolaRotonda implements Cavaliere {
3      private String nome;
4      public CavaliereTavolaRotonda(String nome) {
5          this.nome = nome;
6      }
7      @Override public SantoGraal partiPerMissione() {
8          return new SantoGraal();
9      }
10 }

```

3. Classe Menestrello (Advice)



Java

```

1  // Advice - il Menestrello come Aspect
2  @Component
3  @Aspect
4  public class Menestrello {
5      @Before("execution(*
6          tavola.CavaliereTavolaRotonda.partiPerMissione(..))")
7          public void cantaGesta() {
8              System.out.println("il cavaliere è in partenza..");
9          }
10 }

```

4. Classe ProxyFactoryBean



Java

```

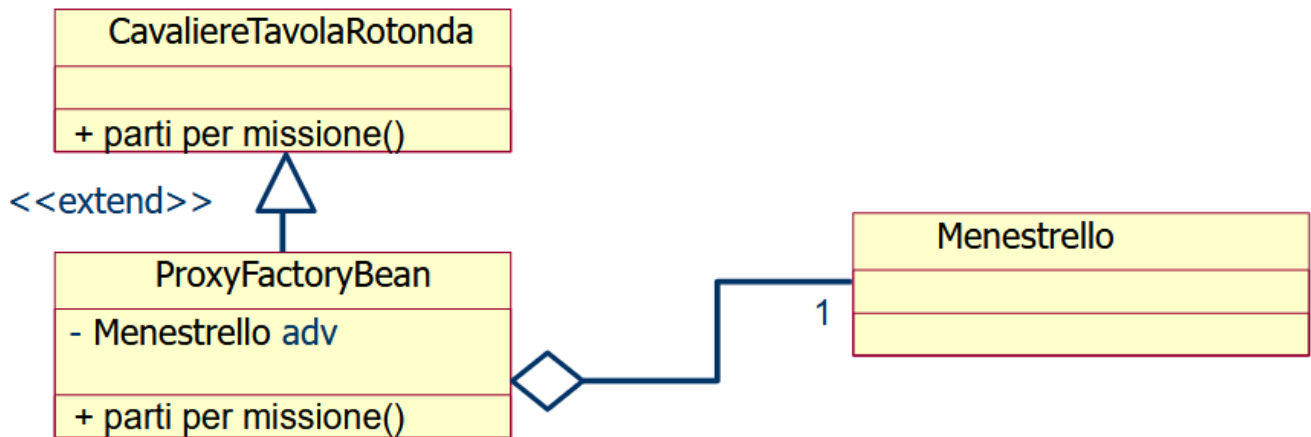
1  // ProxyFactoryBean - implementa la stessa interfaccia
2  public class ProxyFactoryBean implements Cavaliere {
3      private CavaliereTavolaRotonda k; // Target private
4      Menestrello adv; // Advice public
5      ProxyFactoryBean(CavaliereTavolaRotonda k, Menestrello adv) {
6          this.k = k;
7          this.adv = adv;
8      }
9      @Override
10     public SantoGraal partiPerMissione() {
11         adv.cantaGesta(); // @Before - prima della logica di
12         business return k.partiPerMissione(); // delega al Target
13     }
14 }

```

AOP: Il Proxy senza Interfaccia

Il diagramma UML

Questo secondo scenario mostra l'architettura nel caso in cui il target **non implementi una interfaccia** — `CavaliereTavolaRotonda` **è una classe concreta senza contratto.**



In questo caso la `ProxyFactoryBean` non può implementare una interfaccia comune, quindi Spring adotta una strategia diversa:

- la `ProxyFactoryBean` **estende direttamente** `CavaliereTavolaRotonda` **tramite ereditarietà** (`<<extend>>`).

Le differenze rispetto al diagramma precedente sono due:

- Non esiste più l'interfaccia `Cavaliere` — **non c'è contratto comune tra i due**
- La relazione tra `ProxyFactoryBean` e `CavaliereTavolaRotonda` passa da **realizzazione** a **ereditarietà** — **freccia continua con punta triangolare vuota in UML, che in Java corrisponde a** `extends`
- `ProxyFactoryBean` **aggrega comunque il** `Menestrello` **tramite l'attributo** `-Menestrello adv`, **esattamente come nel caso precedente.**

Il risultato per il client rimane lo stesso:

- vede una** `CavaliereTavolaRotonda` — **in questo caso la sottoclasse proxy — senza sapere di essere intercettato.**

Implementazione in Java

- Classe** `CavaliereTavolaRotonda` (Target)



Java

```
1 // Target - classe concreta, nessuna interfaccia
2 public class CavaliereTavolaRotonda {
3     private String nome;
4     public CavaliereTavolaRotonda(String nome) {
5         this.nome = nome;
6     }
7     public SantoGraal partiPerMissione() {
8         return new SantoGraal();
9     }
10 }
```

2. Classe Menestrello (advice)



Java

```
1 // Advice - rimane identico
2 @Component
3 @Aspect
4 public class Menestrello {
5     @Before("execution(*
6         tavola.CavaliereTavolaRotonda.partiPerMissione(..))")
7     public void cantaGesta() {
8         System.out.println("il cavaliere è in partenza..");
9     }
10 }
```

3. Classe ProxyFactoryMethod



Java

```
1 // ProxyFactoryBean - ESTENDE il Target invece di implementare
  // una interfaccia
2 public class ProxyFactoryBean extends CavaliereTavolaRotonda {
3     private Menestrello adv; // Advice public
4     ProxyFactoryBean(String nome, Menestrello adv) {
5         super(nome); // chiama il costruttore della superclasse
6         this.adv = adv;
7     }
8     @Override
9     public SantoGraal partiPerMissione() {
10         adv.cantaGesta(); // @Before - prima della logica di
    business
}
```



```
11         return super.partiPerMissione(); // delega alla
        superclasse
12     }
13 }
```

Il flusso di esecuzione

Quando il client chiama `partiPerMissione()`, in realtà sta chiamando il metodo della `ProxyFactoryBean` — non direttamente il Cavaliere.

È la `ProxyFactoryBean` a coordinare l'ordine di esecuzione in base al tipo di advice configurato:

1. **invoca** `cantaGesta()` **del Menestrello** — perché il joinpoint è `@Before`
2. **invoca** `partiPerMissione()` **di** `CavaliereTavolaRotonda` — la logica di **business**

Il client non si accorge di nulla.

Questo è il weaving a runtime in azione.

🔗 Questi due casi non sono semplicemente due sfumature progettuali — corrispondono a due **librerie diverse** che Spring usa internamente per creare il proxy a runtime.

La scelta però non spetta a te come sviluppatore:

- è Spring che decide automaticamente in base a come hai progettato il tuo codice.
- Se il target **implementa una interfaccia** → **Spring usa JDK Dynamic Proxy, integrato nativamente in Java nel package** `java.lang.reflect`
- Se il target **non implementa una interfaccia** → **Spring usa CGLIB, una libreria esterna inclusa automaticamente tra le dipendenze di Spring**

La sola scelta progettuale che fai tu è quindi usare o meno una interfaccia.

Tutto il resto è gestito da Spring in modo trasparente.

🔔 Un dettaglio pratico importante:

CGLIB ha una limitazione — non può creare proxy di classi o metodi dichiarati `final`, perché non può estendere ciò che è

bloccato all'ereditarietà.

Questa è un'ulteriore ragione per cui programmare verso le interfacce rimane la best practice in Spring.

Ecco la sezione riscritta:

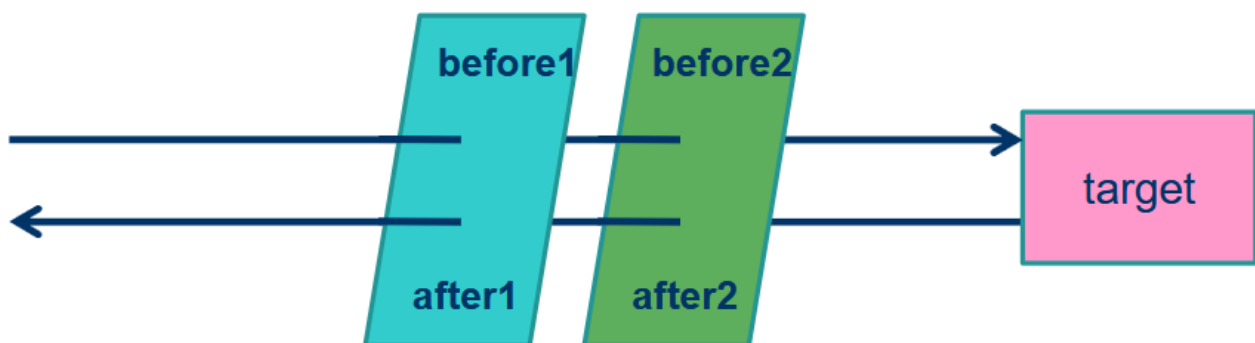
AOP: Gli Intercettori come Filtri

Il meccanismo di filtraggio

È possibile applicare più intercettori in cascata sullo stesso target — esattamente come i filtri di un rubinetto:

- **puoi montare un filtro per il calcare, e subito dopo un secondo filtro per altre impurità. L'acqua passa attraverso entrambi in sequenza prima di arrivare a destinazione.**

Il diagramma mostra esattamente questo:



- **La chiamata del client viaggia verso il target attraversando prima before1 poi before2, raggiunge il target, e al ritorno percorre il cammino inverso — prima after2 poi after1. L'ordine si inverte al ritorno.**

☰ Questo è un dettaglio importante: gli intercettori si comportano come una cipolla:

si avvolgono intorno al target a strati, e si "sbucciano" in ordine inverso al ritorno.

Cosa può fare un intercettore

Ogni intercettore possiede riferimenti diretti a:

- il **target** — la classe intercettata
- il metodo intercettato
- tutti i parametri del metodo

⚠ **Questo significa che l'intercettore non si limita a osservare** — può utilizzare e modificare sia i parametri in ingresso che il valore di ritorno.

In questo senso si dice che l'intercettore filtra il metodo target.

Il limite progettuale

Proprio perché gli intercettori sono così potenti da poter alterare il comportamento del target, vale una regola fondamentale:

- tipicamente un intercettore dovrebbe incapsulare solo funzionalità trasversali che non alterano le funzionalità principali.

Il logging traccia, la sicurezza blocca o lascia passare, le statistiche contano — ma nessuno di questi dovrebbe modificare il risultato di un calcolo o alterare i dati di business.

L'obiettivo dell'AOP è:

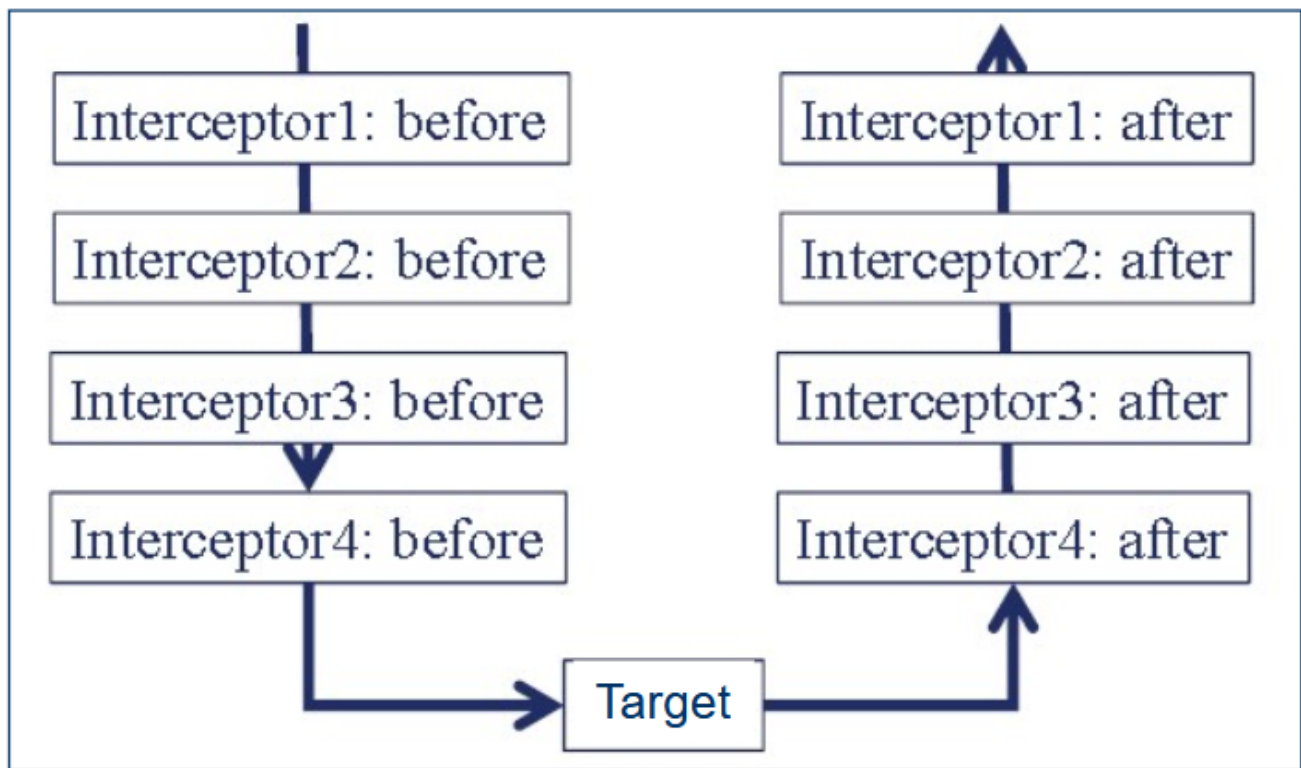
- arricchire il comportamento del sistema senza sporcare o modificare le classi target.

AOP: La Catena di Intercettori (Interceptor Around)

È possibile applicare più advice sullo stesso target, dove ogni advice si occupa di implementare uno specifico aspetto.

In questo caso si parla di **catena di intercettori**.

Il diagramma mostra una catena di 4 intercettori Around.



Il flusso di esecuzione segue questo ordine:

1. Interceptor1: before → Interceptor2: before → Interceptor3: before → Interceptor4: before
2. viene raggiunto il **Target** — la logica di business viene eseguita
3. al ritorno l'ordine si inverte: Interceptor4: after → Interceptor3: after → Interceptor2: after → Interceptor1: after

Questo conferma quanto visto nella sezione precedente: **al ritorno l'ordine si inverte, come sbucciare una cipolla al contrario.**

Due caratteristiche fondamentali di questa architettura:

- **ogni intercettore rappresenta un aspetto che può essere aggiunto o rimosso semplicemente modificando la configurazione** — senza toccare il codice del target né degli altri intercettori
- **gli intercettori sono indipendenti l'uno dall'altro** — ognuno incapsula il proprio concern trasversale senza sapere degli altri

Questa indipendenza è la diretta conseguenza del principio di **Single Responsibility** applicato agli aspetti: **ogni intercettore fa una cosa sola, e la fa bene**

AOP: Configurazione per Spring Boot

Per utilizzare il modulo AOP di Spring è necessario aggiungere due dipendenze al `pom.xml` :

```
</> XML

1  <dependency>
2      <groupId>org.springframework</groupId>
3      <artifactId>spring-aop</artifactId>
4  </dependency>
5
6  <dependency>
7      <groupId>org.aspectj</groupId>
8      <artifactId>aspectjweaver</artifactId>
9  </dependency>
```

La prima dipendenza fornisce il modulo AOP di Spring.

La seconda — `aspectjweaver` — è la libreria di AspectJ:

- **che Spring usa internamente per interpretare le annotazioni e realizzare il weaving a runtime.**

AOP via Annotazioni

Le configurazioni degli intercettori si realizzano mediante **annotazioni**.

Le annotazioni disponibili nel modulo AOP di Spring sono:

- `@Aspect` — **dichiara la classe come intercettore al motore AOP**
- `@Before` — **advice che scatta prima dell'esecuzione del target**
- `@After` — **advice che scatta dopo, indipendentemente dall'esito**
- `@AfterReturning` — **advice che scatta dopo, solo in caso di successo**
- `@AfterThrowing` — **advice che scatta dopo, solo in caso di eccezione**
- `@Around` — **advice che scatta sia prima che dopo**

Le annotazioni da `@Before` ad `@Around` dichiarano il tipo di joinpoint e quindi il tipo di intercettore.

Ciascuna di esse richiede la definizione di un **pointcut** — **l'espressione che individua i metodi target su cui l'advice deve agire.**

- `@Pointcut` — **centralizza la dichiarazione del pointcut in un unico punto, evitando che ogni singolo joinpoint la ridichiari esplicitamente. È utile quando lo stesso pointcut viene riutilizzato da più advice.**

AOP: Esempio con Più Joinpoint



Java

```
1  @Aspect
2  @Component
3  public class Spettatore {
4
5      @Pointcut("execution(* esempio.Artista.perform(..))")
6      public void performance() {}
7
8      @Before("performance()")
9      public void prendePosto() {
10         System.out.println("Il pubblico prende posto");
11     }
12
13     @Before("performance()")
14     public void spegneCellulari() {
15         System.out.println("Il pubblico spegne i cellulari");
16     }
17
18     @AfterReturning("performance()")
19     public void applaude() {
20         System.out.println("CLAP CLAP CLAP CLAP CLAP");
21     }
22
23     @AfterThrowing("performance()")
24     public void vieneRimborsato() {
25         System.out.println("Lo spettacolo è stato interrotto");
26     }
27 }
```

Questo esempio mostra come un unico **Aspect** — lo **Spettatore** — possa definire **più advice** sullo stesso **target**, ciascuno con il proprio **joinpoint**.

Il punto più importante da notare è l'uso di `@Pointcut`:

- **invece di ripetere la stringa** `execution(* esempio.Artista.perform(..))` **in ogni singola annotazione, la si centralizza in un metodo dedicato chiamato** `performance()`.
- **In questo caso il corpo del metodo è vuoto — non fa nulla a runtime.**
 - **Il suo unico scopo è dare un nome al pointcut, che poi viene richiamato da tutti gli advice tramite** `"performance()"`.

Il flusso di esecuzione sarà quindi:

1. `prendePosto()` e `spegneCellulari()` — eseguiti prima dello spettacolo (`@Before`)
2. `Artista.perform()` — la logica di business
3. `applaude()` — eseguito solo se lo spettacolo va a buon fine (`@AfterReturning`)
4. `vieneRimborsato()` — eseguito solo se viene sollevata una eccezione (`@AfterThrowing`)

AOP: Regole Sintattiche dei Metodi Joinpoint

I metodi che implementano i joinpoint seguono alcune regole precise:

1. **Visibilità:** devono essere dichiarati `public`.
2. **Parametri:** possono avere zero parametri oppure accettare tipi speciali che permettono di accedere al contesto del metodo intercettato:
 - `JoinPoint` — disponibile per tutti i tipi di advice, permette di accedere a metadati sul metodo target come il nome del metodo e i suoi argomenti
 - `ProceedingJoinPoint` — disponibile solo per `@Around`:
 - permette di controllare se e quando far proseguire l'esecuzione del target tramite il metodo `proceed()`
3. **Tipo di ritorno:** generalmente è `void`.
 - l'intercettore non dovrebbe restituire un valore proprio, ma limitarsi a realizzare un servizio trasversale.
 - L'unica eccezione è `@Around`, dove si valuta se restituire il valore originale del target oppure un valore diverso deciso dall'intercettore stesso.

AOP: Advice di tipo Around

L'advice `@Around` permette di:

- sostituire tutti i singoli advice `@Before`, `@AfterReturning` e `@AfterThrowing` con un unico metodo che gestisce l'intero ciclo di esecuzione.



Java

```
1 @Around("performance()")
```

```

2  public void guardaPerformance(ProceedingJoinPoint joinpoint) {
3      try {
4          // pre-processing - equivalente a @Before
5          System.out.println("Il pubblico prende posto");
6          System.out.println("Il pubblico spegne i cellulari");
7          long start = System.currentTimeMillis();
8
9          // invocazione al prossimo advice oppure al metodo target
10         joinpoint.proceed();
11
12         // post-processing - equivalente a @AfterReturning
13         long end = System.currentTimeMillis();
14         System.out.println("CLAP CLAP CLAP CLAP CLAP");
15         System.out.println("La performance è durata " + (end -
16             start) + " millisecondi.");
17     } catch (Throwable t) {
18         // equivalente a @AfterThrowing
19         System.out.println("Lo spettacolo è stato interrotto!");
20     }
21 }

```

Il ruolo di ProceedingJoinPoint

Il metodo accetta come parametro un oggetto `ProceedingJoinPoint` e restituisce `void` oppure `Object`.

Il `ProceedingJoinPoint` è:

- il meccanismo con cui l'advice `@Around` cede il controllo al passo successivo della catena tramite `joinpoint.proceed()`.

⚠ Un dettaglio importante :

abbiamo detto che il metodo invocato `.proceed()` cede il controllo al "passo successivo" della catena.

Questo però non è necessariamente il Target — se nella catena ci sono più intercettori, `joinpoint.proceed()` invoca il prossimo advice nella catena, non direttamente il metodo target.

Solo l'ultimo intercettore della catena chiamerà effettivamente il target.

Questo è esattamente il meccanismo "a cipolla" che abbiamo visto nella sezione sulla catena di intercettori.

Il flusso in sintesi

Tutto ciò che sta **prima** di `joinpoint.proceed()` equivale a `@Before`, tutto ciò che sta **dopo** equivale a `@AfterReturning`, e il blocco `catch` equivale a `@AfterThrowing` — il tutto in un unico metodo coeso.

AOP: L'oggetto `ProceedingJoinPoint`

`ProceedingJoinPoint` è:

- l'oggetto che modella il prossimo elemento nella catena degli intercettori, oppure il target stesso se ci troviamo all'ultimo anello della catena.

Invocando `joinpoint.proceed()` si procede in avanti nella catena **senza sapere esattamente cosa c'è al passo successivo** — potrebbe essere un altro intercettore o il target finale, come abbiamo già anticipato.

Questo è il meccanismo che garantisce l'indipendenza tra gli elementi della catena:

- ogni intercettore non conosce i suoi vicini, sa solo che può cedere il controllo al passo successivo.

`ProceedingJoinPoint` mette inoltre a disposizione metodi utili per recuperare informazioni sul metodo target intercettato:

- `getSignature().getName()` — restituisce il nome del metodo target
- `getTarget()` — restituisce la classe proxy del metodo target

Questi metodi tornano particolarmente utili nei servizi di logging, dove si vuole tracciare quale metodo è stato chiamato e su quale classe.

❓ Come si fa a sapere qual è il prossimo elemento della catena?

La risposta breve è: **non lo sai, e non devi saperlo.**

Questo è esattamente il punto di forza del meccanismo.

Quando si chiama `joinpoint.proceed()` si sta dicendo a Spring "vai avanti" — ma non interessa dove.

Potrebbe essere un altro intercettore, potrebbe essere il target finale.

Questa ignoranza è voluta e progettata.

🔗 È lo stesso principio che abbiamo già visto con la Dependency Injection:

- la `OperazioneController` non sa quale implementazione concreta di `OperazioneService` riceve — sa solo che ha un oggetto che rispetta quel contratto.

Qui è uguale:

- ogni intercettore sa solo che c'è un "passo successivo", non chi è.

È Spring che conosce l'intera catena e gestisce l'ordine internamente.

Ad esempio, Spring sa che dopo `Interceptor1` viene `Interceptor2`, e dopo `Interceptor2` viene il target.

Lo sviluppatore vede solo il suo pezzo di catena.

L'unico modo per influenzare questo ordine è `@Order` — ma anche lì non si sta dicendo "dopo di me viene X", ma si sta solo dicendo "io sono il numero 11" e si lascia a Spring il compito di assemblare la sequenza corretta.

Questo è il **Low Coupling** in azione: ogni intercettore è completamente indipendente dagli altri.

AOP: Ordine degli Intercettori

Quando più intercettori agiscono sullo stesso target, Spring segue un ordine preciso per gli advice dello stesso tipo:

1. prima gli intercettori `@Before` — in ordine casuale tra loro
2. poi gli eventuali `@Around` — in ordine casuale tra loro
3. poi gli intercettori `@After` — in ordine casuale tra loro

Se si hanno advice diversi applicati allo stesso target, l'ordine complessivo non è garantito.

Per controllarlo esplicitamente si usa l'annotazione `@Order` posizionata sulla classe advice:



Java

```
1  @Aspect
2  @Component
3  @Order(value = 11)
4  public class PrimoAspect { ... }
5
6  @Aspect
7  @Component
8  @Order(value = 22)
9  public class SecondoAspect { ... }
10
11 @Aspect
12 @Component
13 @Order(value = 33)
14 public class TerzoAspect { ... }
```

I valore numerico stabilisce la sequenza — più basso è il numero, prima parte l'intercettore.

I valori non devono essere necessariamente consecutivi:

- ****quello che conta è la loro sequenza crescente.****



Gli intercettori senza `@Order` partiranno sempre in coda a quelli che ce l'hanno, in ordine casuale tra loro.

AOP: Il Linguaggio dei Pointcut

Spring AOP offre una sintassi dedicata per la definizione dei pointcut, basata su un insieme di **operatori** che permettono di specificare con precisione quali metodi intercettare.

Operatore	Descrizione
<code>execution()</code>	Definisce un joinpoint come l'esecuzione di un metodo.
<code>args()</code>	Definisce un joinpoint i cui argomenti sono di uno specifico tipo.
<code>target()</code>	Definisce un joinpoint dove l'oggetto target è di un tipo specifico.
<code>within()</code>	Limita un joinpoint all'interno di specifici package.

L'operatore principale è `execution()` — è quello che abbiamo già usato in tutti gli esempi precedenti. Gli altri operatori servono a raffinare ulteriormente la selezione, ma spesso vengono usati in combinazione con `execution()`.

Esempio 1 — Intercettare un metodo specifico



Java

```
1 execution(* esempio.Strumento.suona(..))
```

Analizziamo la sintassi pezzo per pezzo:

- `execution` — indica che stiamo intercettando l'esecuzione di un metodo
- `*` — il tipo di ritorno è qualunque
- `esempio.Strumento` — il nome completo della classe (o interfaccia), e del package dove risiede quella classe, che possiede il metodo
- `suona` — il nome del metodo da intercettare
- `(..)` — qualunque lista di argomenti

Questo pointcut intercetta quindi qualsiasi chiamata al metodo `suona()` della classe `Strumento` nel package `esempio`, indipendentemente dal tipo di ritorno e dai parametri.

Esempio 2 — Combinare operatori con `&&`



Java

```
1 execution(* *.Strumento.suona(..)) && within(esempio.*)
```

Qui entrano in gioco due operatori combinati con l'operatore logico `&&`:

- `execution(* *.Strumento.suona(..))` — intercetta il metodo `suona()` su qualsiasi classe `Strumento` in qualsiasi package (`*`)
- `within(esempio.*)` — limita l'intercettazione alle sole classi contenute nel package `esempio` e in tutti i suoi sotto-pacchetti

Il risultato è che il metodo `suona()` viene intercettato solo se la classe `Strumento` appartiene al package `esempio` o a uno dei suoi sotto-package.

È possibile combinare gli operatori usando la stessa logica booleana di Java:

- `&&` (and),
- `||` (or)
- `e !` (not)

Permettendo di costruire pointcut molto precisi e selettivi.

AOP: Altri Esempi di Pointcut

Esempi con `execution()`

Qualsiasi metodo pubblico:



Java

```
1 execution(public * *(..))
```

- Il primo `*` indica qualsiasi tipo di ritorno,
- il secondo `*` indica qualsiasi classe e qualsiasi metodo.

Qualsiasi metodo che inizia con "set":



Java

```
1 execution(* set*(..))
```

L'asterisco dopo `set` funziona come wildcard — intercetta `setName()`, `setEtà()`, `setIndirizzo()` e qualsiasi altro setter, su qualsiasi classe.

Qualsiasi metodo di una classe o interfaccia specifica:



Java

```
1 execution(* com.xyz.service.AccountService.*(..))
```

L' `*` finale indica qualsiasi metodo di `AccountService` — utile quando vuoi applicare un aspect all'intera superficie di una classe senza specificare i singoli metodi.

Qualsiasi metodo del package `service` :



Java

```
1 execution(* com.xyz.service.*.*(..))
```

Il primo `*` dopo `service.` indica qualsiasi classe del package, il secondo `*` indica qualsiasi metodo di quella classe.

Qualsiasi metodo del package `service` e dei suoi sotto-package:



Java

```
1 execution(* com.xyz.service..*.*(..))
```

Il doppio punto `..` è la sintassi che indica "questo package e tutti i suoi sotto-package" — una differenza sottile ma importante rispetto all'esempio precedente.

Esempi con `within()` e `args()`

Qualsiasi metodo all'interno del package `service` :



Java

```
1 within(com.xyz.service.*)
```

Qualsiasi metodo all'interno del package `service` e dei suoi sottopackage:



Java

```
1 within(com.xyz.service..*)
```

Anche qui il doppio punto `..` estende la selezione ai sottopackage.

Qualsiasi metodo che accetta un parametro di tipo `Serializable` :



Java

```
1 args(java.io.Serializable)
```

Questo operatore non guarda il nome del metodo né la classe — **guarda esclusivamente il tipo del parametro in ingresso** .

Utile quando vuoi intercettare tutti i metodi che ricevono un certo tipo di dato, indipendentemente da dove si trovano.

- ☞ Queste espressioni possono essere combinate con `&&`, `||` e `!` esattamente come negli esempi precedenti, permettendo di costruire pointcut molto precisi che selezionano esattamente i metodi che ti interessano.